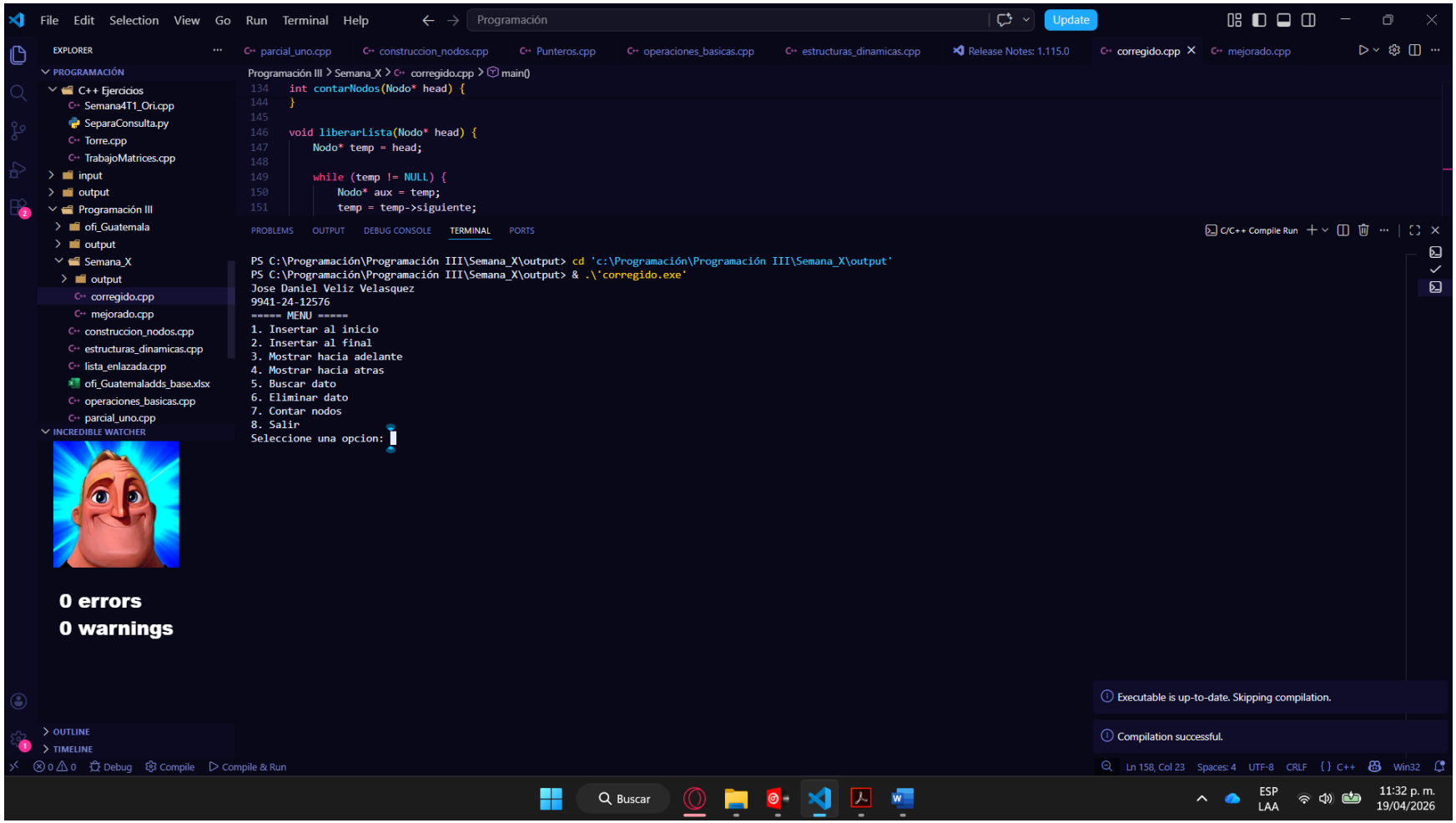
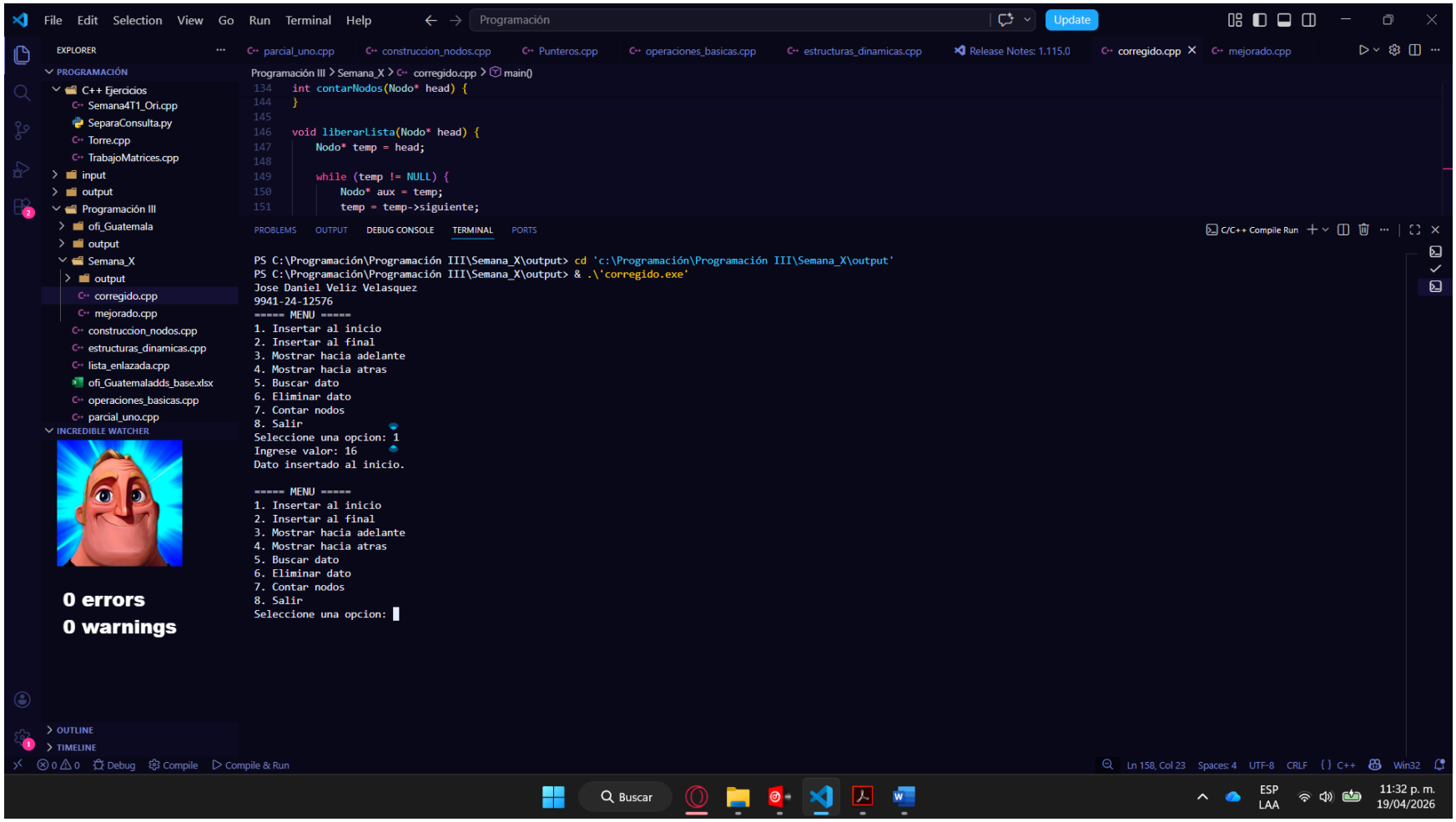
José Daniel Véliz Velásquez

9941-24-12576



- prueba de inserción,



File Edit Selection View Go Run Terminal Help

Programación

Update

EXPLORER

PROGRAMACIÓN

- C++ Ejercicios
  - Semana4T1_Ori.cpp
  - SeparaConsulta.py
  - Torre.cpp
  - TrabajoMatrices.cpp
- input
- output
- Programación III
  - ofi_Guatemala
  - output
  - Semana_X
    - output
    - corregido.cpp
    - mejorado.cpp
  - construccion_nodos.cpp
  - estructuras_dinamicas.cpp
  - lista_enlazada.cpp
  - ofi_Guatemaladds_base.xlsx
  - operaciones_basicas.cpp
  - parcial_uno.cpp

INCREDIBLE WATCHER

0 errors

0 warnings

OUTLINE

TIMELINE

parcial_uno.cpp | construccion_nodos.cpp | Punteros.cpp | operaciones_basicas.cpp | estructuras_dinamicas.cpp | Release Notes: 1.115.0 | corregido.cpp | mejorado.cpp

Programación III > Semana_X > corregido.cpp > main()

```cpp
int contarNodos(Nodo* head) {
}

void liberarLista(Nodo* head) {
    Nodo* temp = head;

    while (temp != NULL) {
        Nodo* aux = temp;
        temp = temp->siguiente;
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

C/C++ Compile Run

```
9941-24-12576
===== MENU =====
1. Insertar al inicio
2. Insertar al final
3. Mostrar hacia adelante
4. Mostrar hacia atras
5. Buscar dato
6. Eliminar dato
7. Contar nodos
8. Salir
Seleccione una opcion: 1
Ingrese valor: 16
Dato insertado al inicio.

===== MENU =====
1. Insertar al inicio
2. Insertar al final
3. Mostrar hacia adelante
4. Mostrar hacia atras
5. Buscar dato
6. Eliminar dato
7. Contar nodos
8. Salir
Seleccione una opcion: 2
Ingrese valor: 30
Dato insertado al final.

===== MENU =====
1. Insertar al inicio
2. Insertar al final
3. Mostrar hacia adelante
4. Mostrar hacia atras
5. Buscar dato
6. Eliminar dato
7. Contar nodos
8. Salir
Seleccione una opcion:
```

0 0 Debug Compile Compile & Run

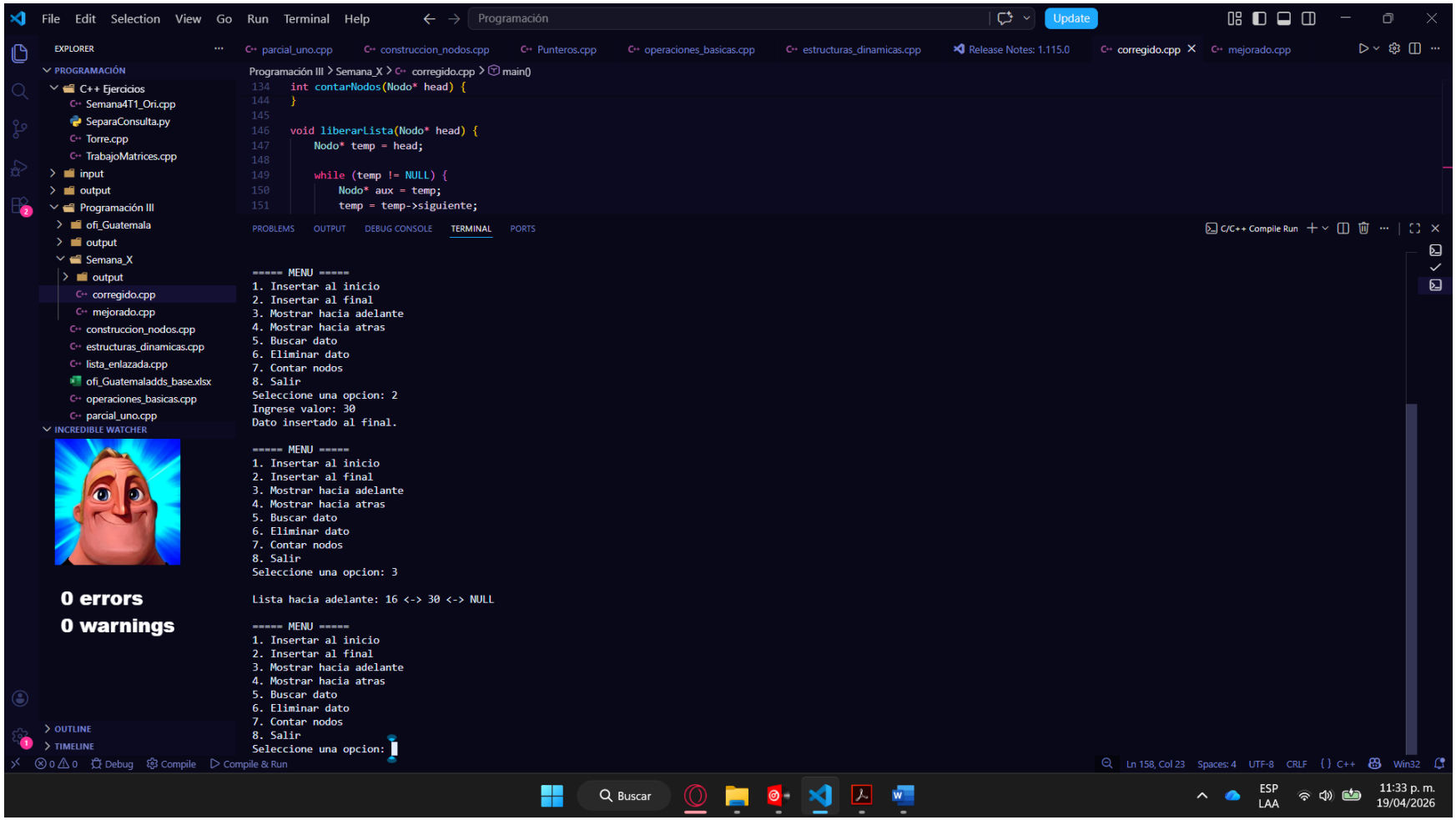
Ln 158, Col 23 Spaces: 4 UTF-8 CRLF {} C++ Win32

Buscar

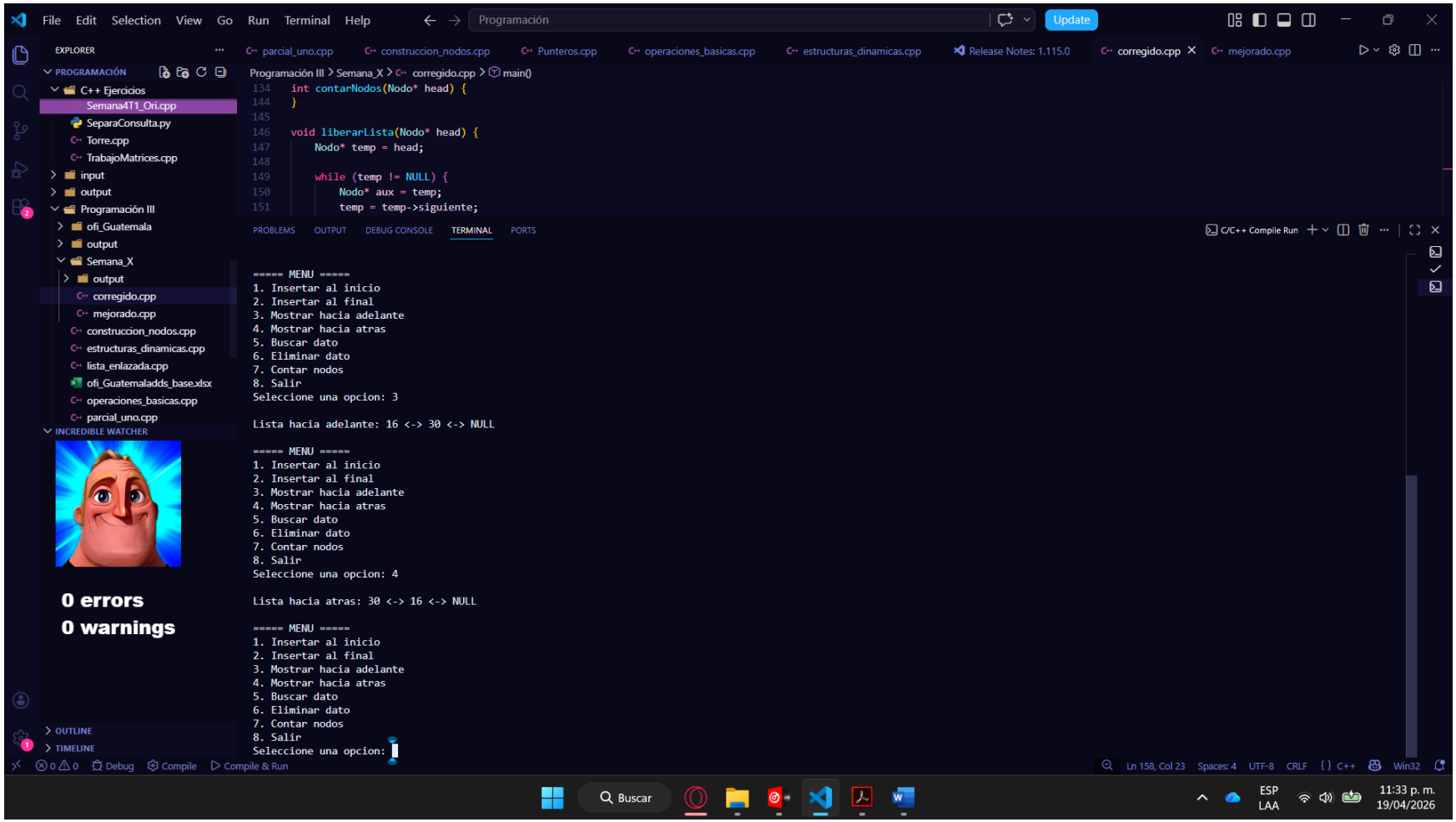
ESP LAA

11:32 p. m. 19/04/2026

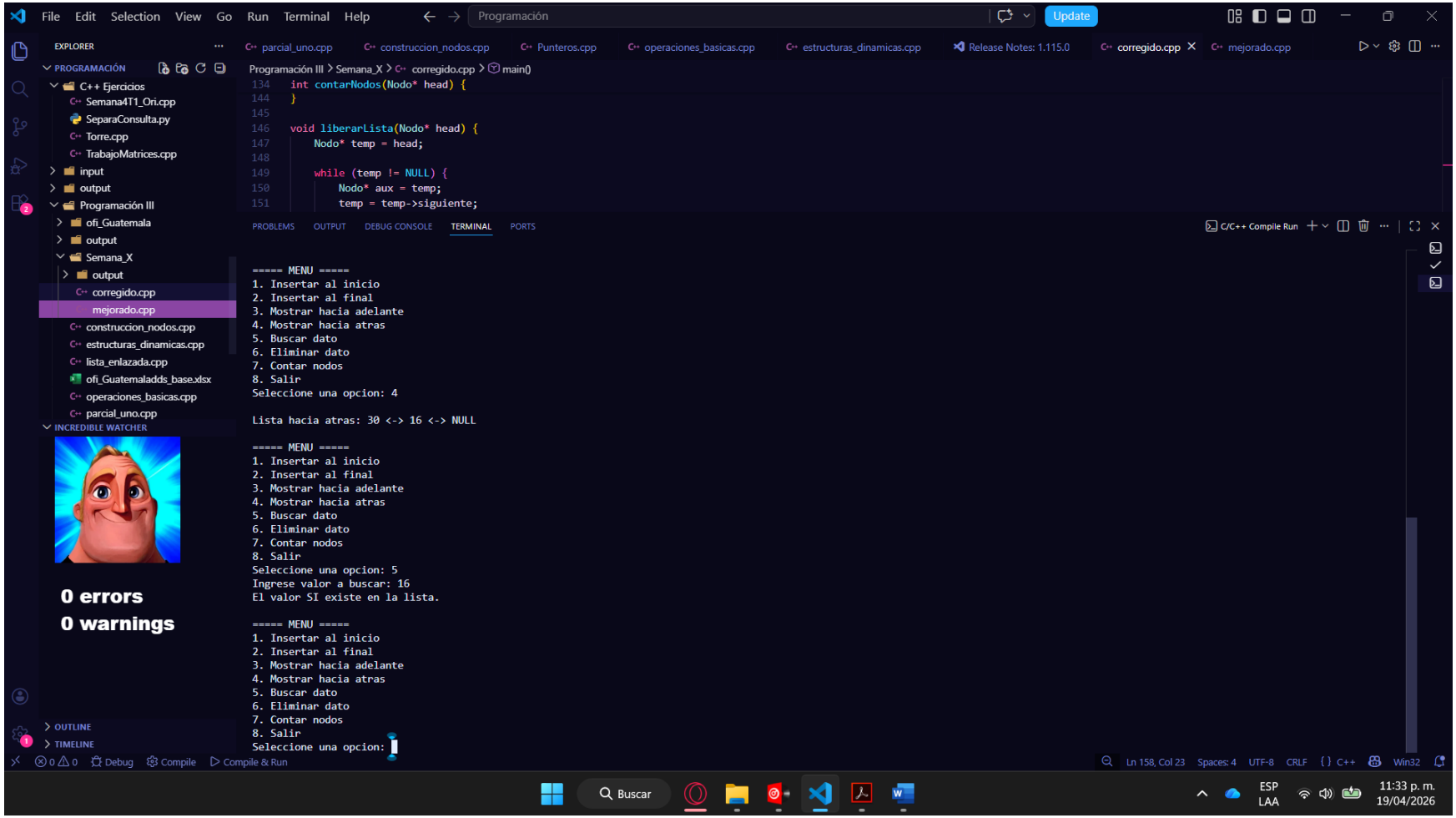
- prueba de recorrido hacia adelante,



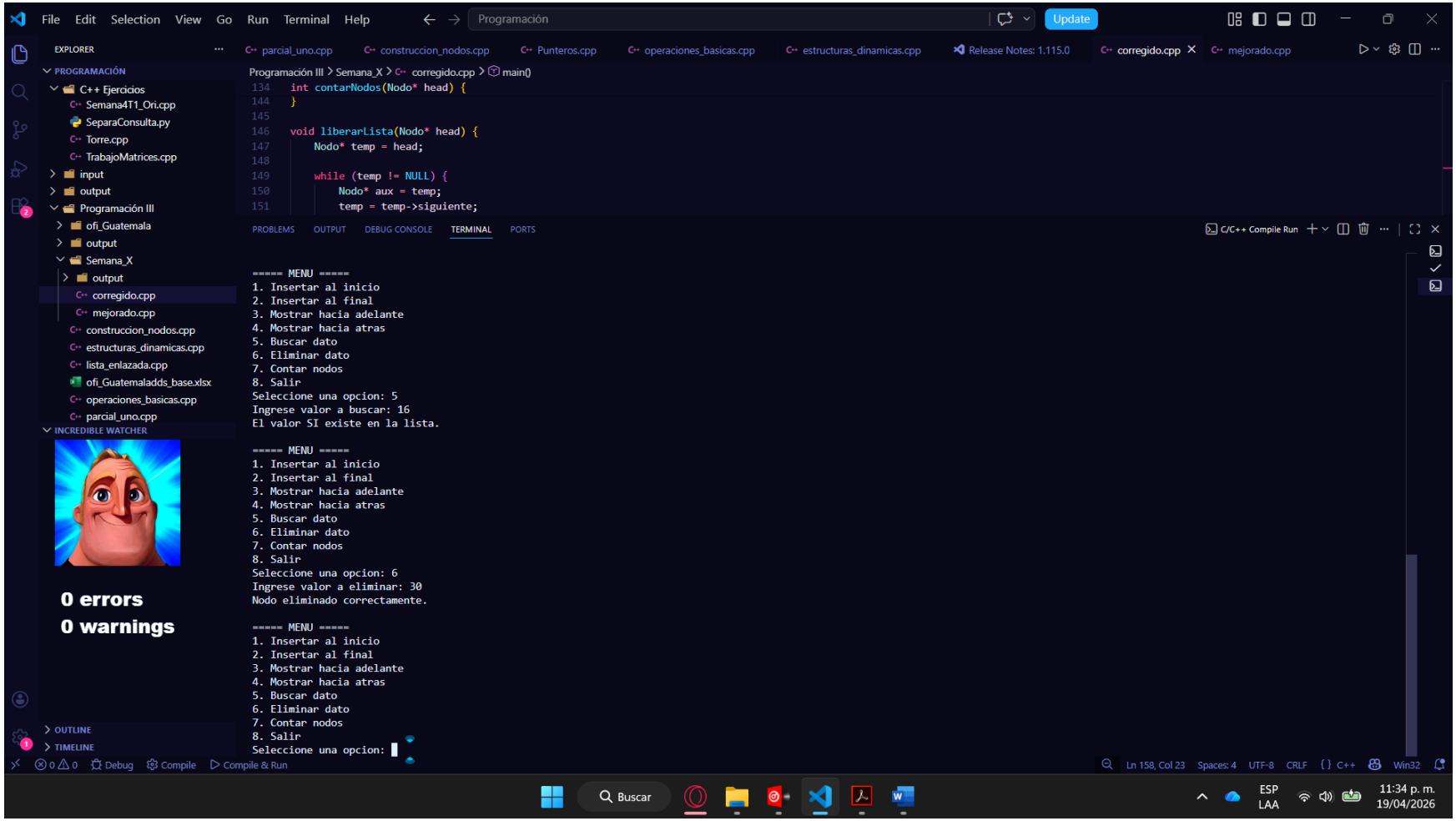
- prueba de recorrido hacia atrás,



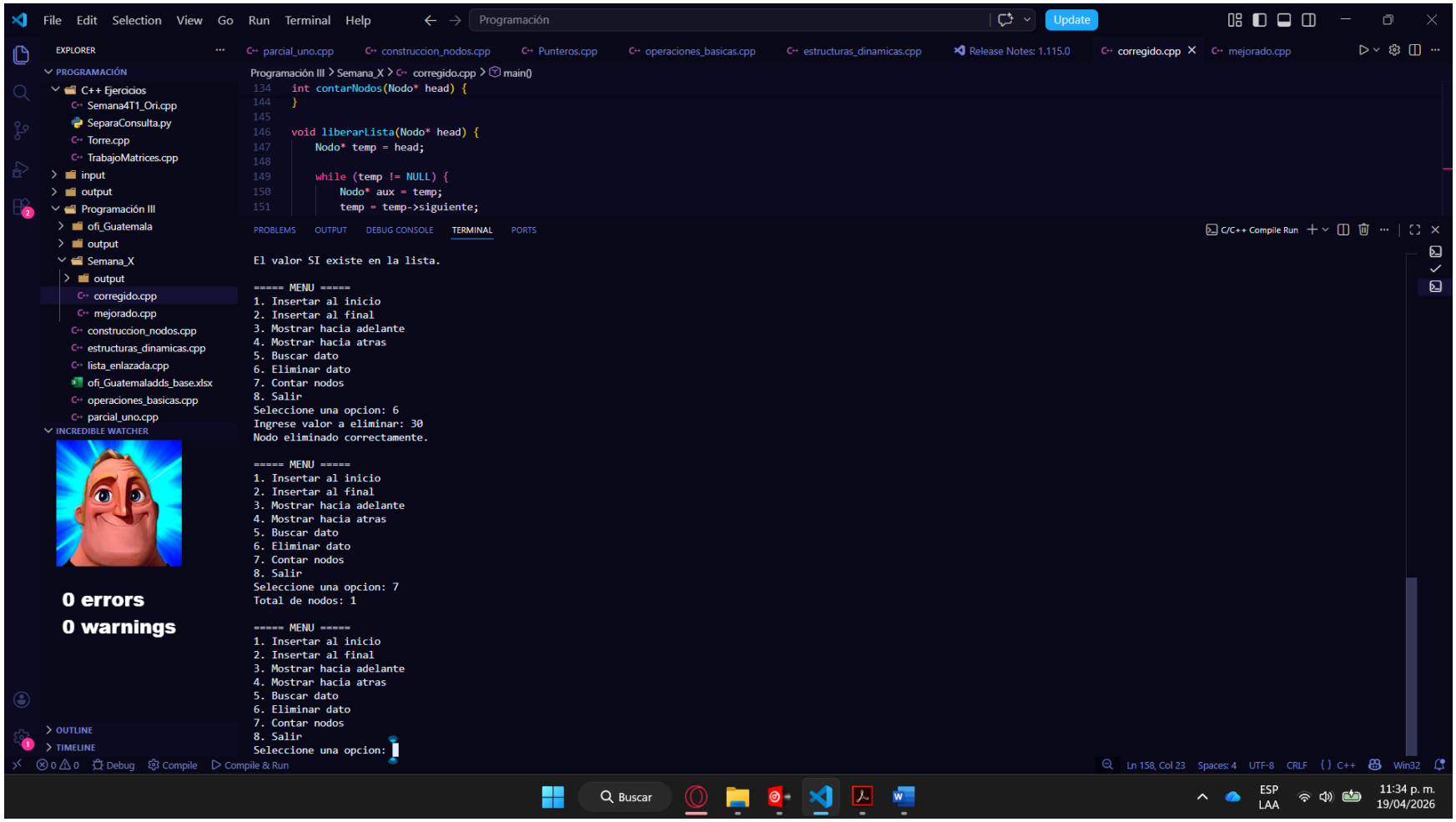
- prueba de búsqueda,



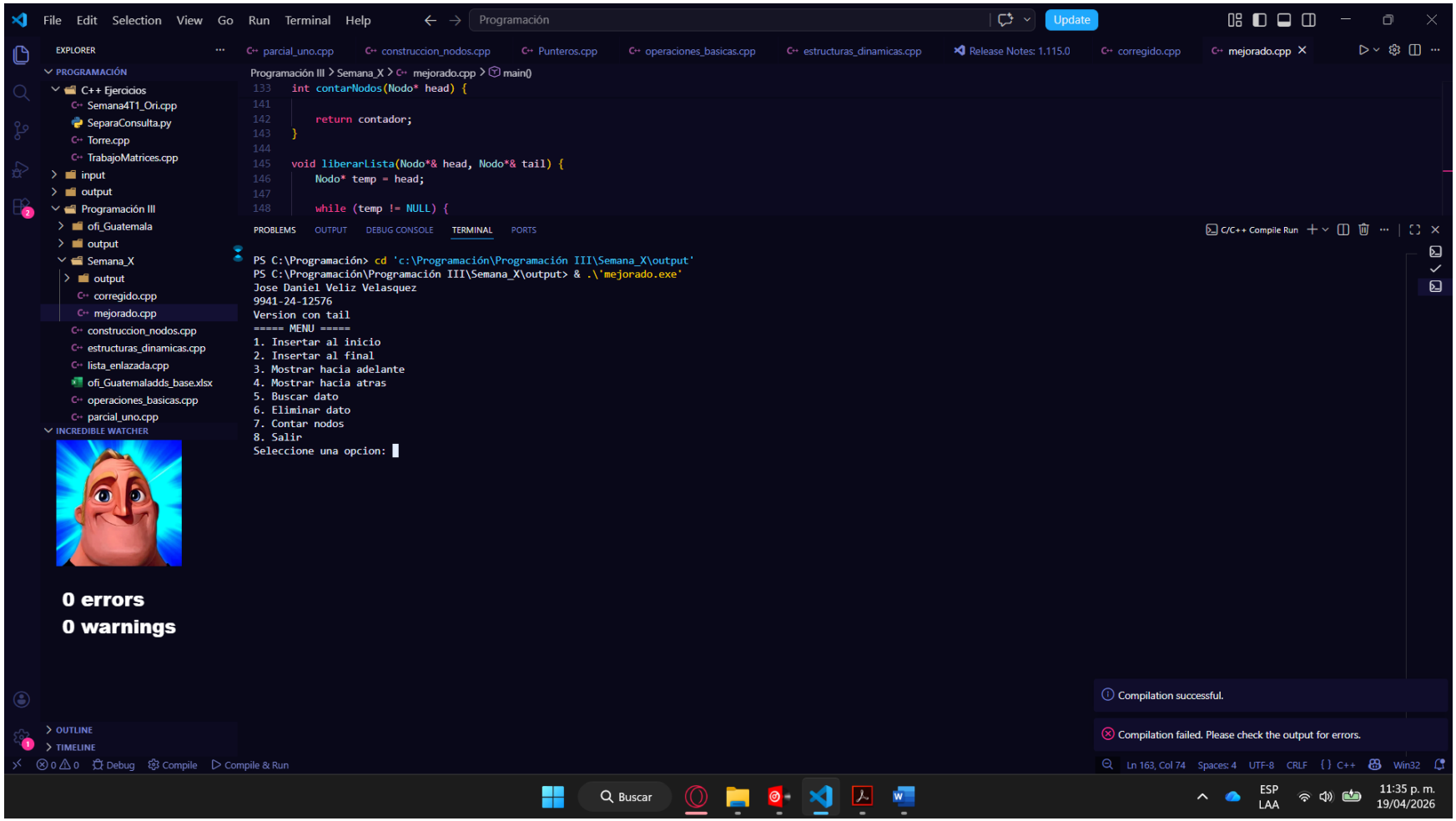
- prueba de eliminación,



- prueba de conteo de nodos,

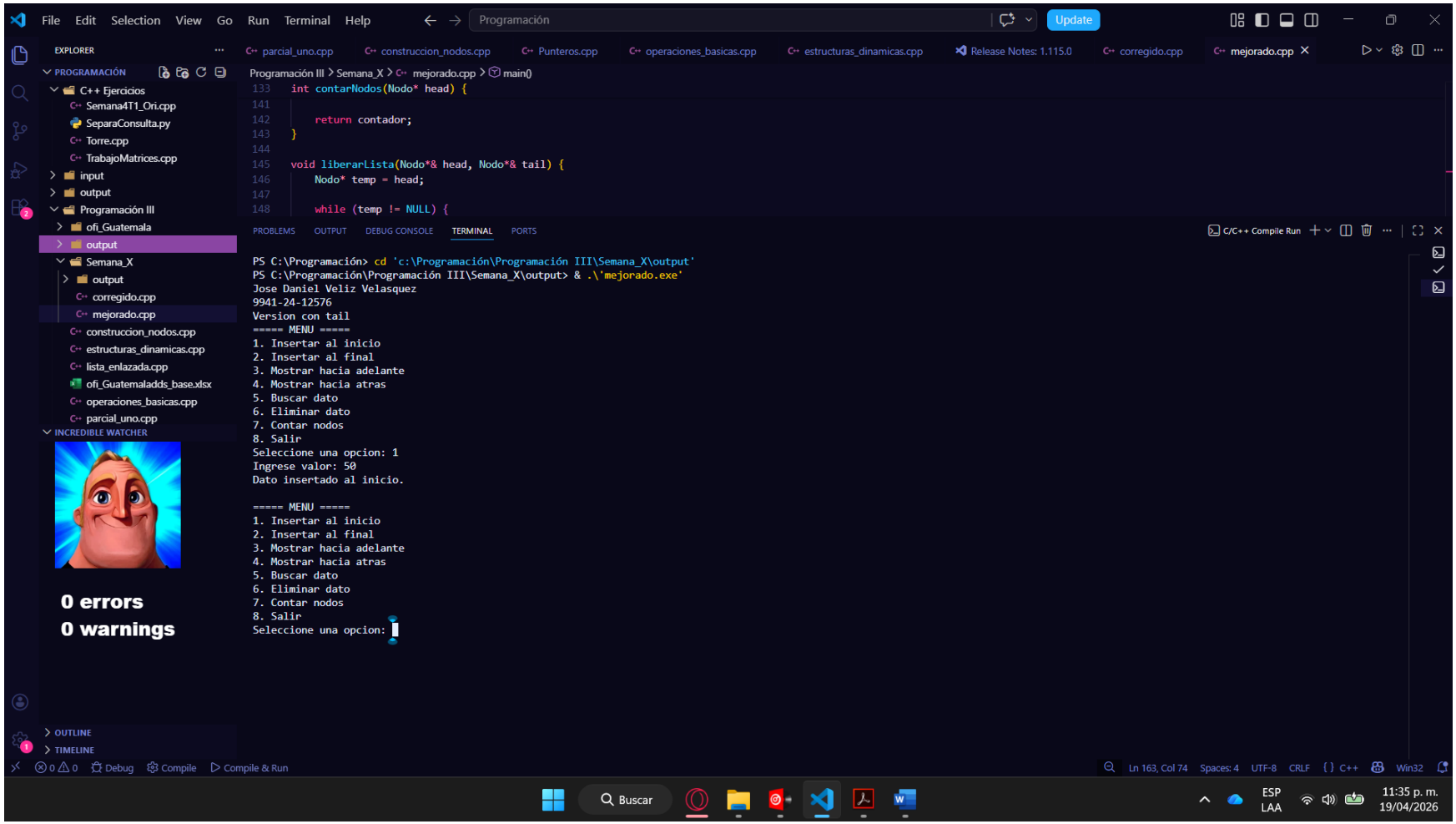


- prueba de funcionamiento de la versión con `tail`.



,

- prueba de inserción,



File Edit Selection View Go Run Terminal Help

Programación

Update

EXPLORER

PROGRAMACIÓN

- C++ Ejercicios
  - Semana4T1_Ori.cpp
  - SeparaConsulta.py
  - Torre.cpp
  - TrabajoMatrices.cpp
  - input
  - output
  - Programación III
    - ofi_Guatemala
    - output
    - Semana_X
      - output
      - corregido.cpp
      - mejorado.cpp
      - construccion_nodos.cpp
      - estructuras_dinamicas.cpp
      - lista_enlazada.cpp
      - ofi_Guatemaladds_base.xlsx
      - operaciones_basicas.cpp
      - parcial_uno.cpp

INCREDIBLE WATCHER

0 errors

0 warnings

OUTLINE

TIMELINE

parcial_uno.cpp | construccion_nodos.cpp | Punteros.cpp | operaciones_basicas.cpp | estructuras_dinamicas.cpp | Release Notes: 1.115.0 | corregido.cpp | mejorado.cpp

Programación III > Semana_X > mejorado.cpp > main()

```cpp
int contarNodos(Nodo* head) {

    return contador;
}

void liberarLista(Nodo*& head, Nodo*& tail) {
    Nodo* temp = head;

    while (temp != NULL) {
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

C/C++ Compile Run

```
Version con tail
===== MENU =====
1. Insertar al inicio
2. Insertar al final
3. Mostrar hacia adelante
4. Mostrar hacia atras
5. Buscar dato
6. Eliminar dato
7. Contar nodos
8. Salir
Seleccione una opcion: 1
Ingrese valor: 50
Dato insertado al inicio.

===== MENU =====
1. Insertar al inicio
2. Insertar al final
3. Mostrar hacia adelante
4. Mostrar hacia atras
5. Buscar dato
6. Eliminar dato
7. Contar nodos
8. Salir
Seleccione una opcion: 2
Ingrese valor: 10
Dato insertado al final.

===== MENU =====
1. Insertar al inicio
2. Insertar al final
3. Mostrar hacia adelante
4. Mostrar hacia atras
5. Buscar dato
6. Eliminar dato
7. Contar nodos
8. Salir
Seleccione una opcion:
```

0 0 Debug Compile Compile & Run

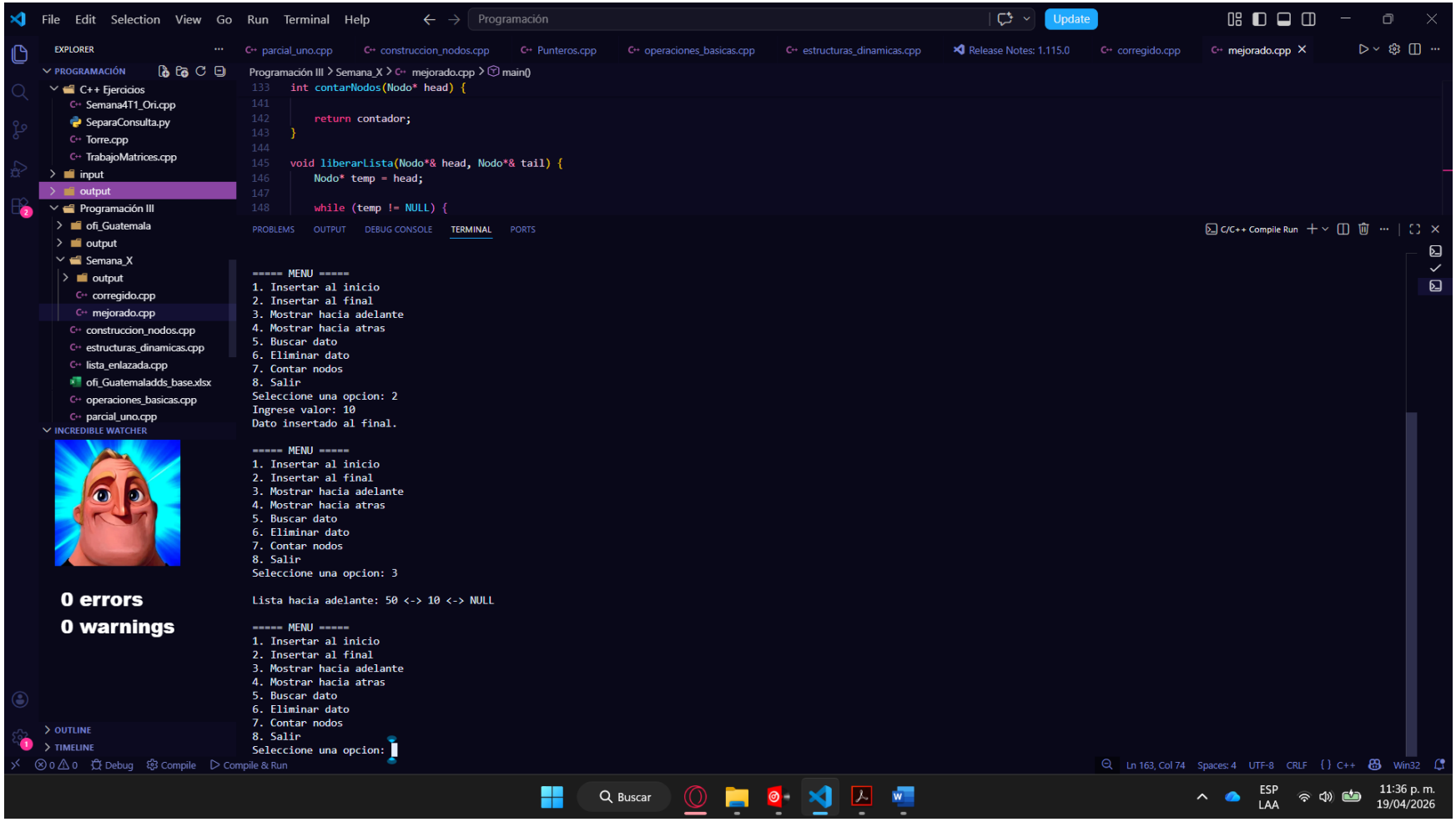
Ln 163, Col 74 Spaces: 4 UTF-8 CRLF C++ Win32

Buscar

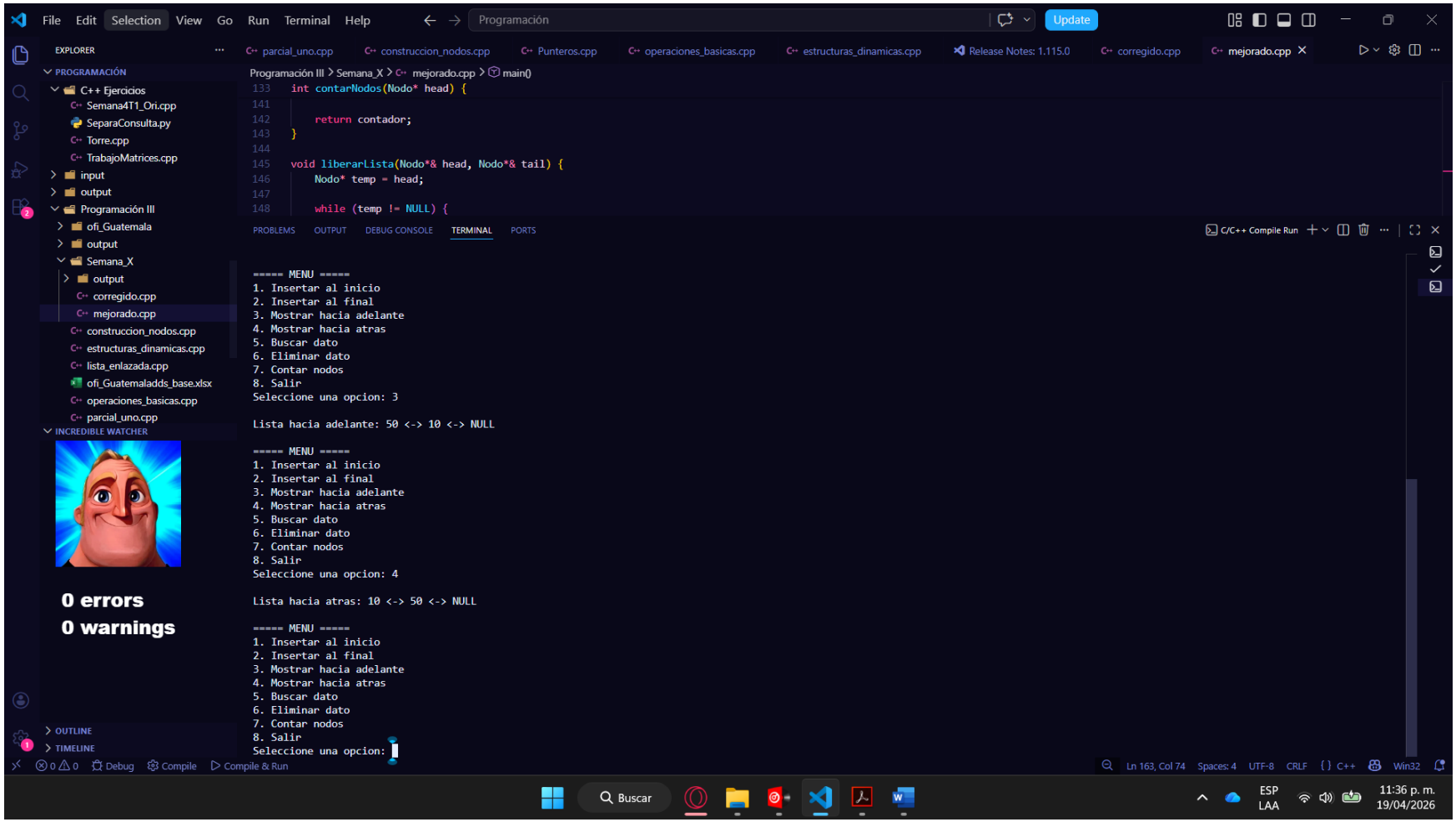
ESP LAA

11:35 p. m. 19/04/2026

- prueba de recorrido hacia adelante,



- prueba de recorrido hacia atrás,



Comparación entre ambas versiones

1. ¿Qué errores presentaba el código original?

El código original tenía varios errores básicos que hacían que no funcionara bien. Por ejemplo, usaba = en lugar de == en condiciones, lo cual hacía comparaciones incorrectas. También había problemas al enlazar los nodos, especialmente al insertar y eliminar, donde algunos punteros quedaban mal conectados. Además, en algunos recorridos se usaban punteros incorrectos (anterior en lugar de siguiente), lo que provocaba que la lista no se mostrara correctamente.

2. ¿Qué problemas tenía la versión sin tail al recorrer hacia atrás?

El principal problema era que no se tenía acceso directo al último nodo. Entonces, para recorrer hacia atrás, primero había que recorrer toda la lista desde el inicio hasta el final. Esto hacía el proceso más lento y poco eficiente, especialmente si la lista era grande.

3. ¿Qué ventaja aporta usar tail?

Usar tail facilita mucho el trabajo porque siempre tenemos acceso directo al último nodo. Esto permite insertar al final de forma rápida y también recorrer la lista hacia atrás sin tener que hacer un recorrido previo. En general, hace el programa más eficiente y más fácil de manejar.

4. ¿Qué operaciones mejoran con esta modificación?

Principalmente mejora la inserción al final, ya que antes había que recorrer toda la lista y ahora se hace directamente. También mejora el recorrido hacia atrás, porque ya no hay que buscar el último nodo, sino que se empieza desde tail. Las demás operaciones siguen funcionando igual, pero el código queda más optimizado.

5. ¿Qué cuidados adicionales deben tenerse al eliminar nodos cuando existe tail?

Hay que tener cuidado de mantener actualizados los punteros head y tail. Por ejemplo, si se elimina el último nodo, hay que mover tail al nodo anterior. Si se elimina el primero, hay que actualizar head. Y si solo hay un nodo y se elimina, ambos deben quedar en NULL. Si no se controla esto, la lista puede quedar mal enlazada o provocar errores.